Document number:	P0944R0
Date:	2018-02-11
Project:	C++ Extensions for Ranges, Library Evolution Working Group
Reply-to:	Casey Carter <Casey@Carter.net>

Contiguous Ranges

Synopsis

This paper proposes extensions to ISO/IEC TS 21425:2017 “C++ Extensions for ranges” (The “Ranges TS”) to enable first-class support for contiguous iterators and contiguous ranges. Contiguity of storage is a critical property of data which the Ranges design should be able to communicate to generic algorithms.

Background

The C++ standard library has been slowly recognizing the increasing importance of data contiguity. The word “contiguous” does not appear in the C++98 standard library specification. Nevertheless:

- `valarray` was implicitly required to store elements contiguously per [lib.valarray.access]/3:

  > The expression `&a[i + j] == &a[i] + j` evaluates as `true` for all `size_t i` and `size_t j` such that `i+j` is less than the length of the non-constant array `a`."

- In those days a quality implementation was expected to implement `vector` (not `vector<bool>`) using contiguous memory despite that property not being required. Even in the “dark ages” of the late 1990s we understood the interaction of contiguous storage with caches; hence the exhortation in [lib.sequence.reqmts]/2 “`vector` is the type of sequence that should be used by default.”

As a Technical Corrigendum, C++03 made very few changes. One of those changes was to promote the contiguity of `vector<not_bool>` from QoI to a requirement in [lib.vector]/1:

> The elements of a vector are stored contiguously, meaning that if `v` is a `vector<T, Allocator>` where `T` is some type other than `bool`, then it obeys the identity `&v[n] == &v[0] + n` for all `0 <= n < v.size()`. Presumably five more years of studying the interactions of contiguity with caching made it clear to WG21 that contiguity needed to be mandated and non-contiguous `vector` implementation should be clearly banned.

Eight more years of processor speed increasing faster than memory latency made cache effects even more pronounced in the C++11 era. The standard library responded by adding more support for contiguity: `basic_string` added a requirement for contiguous storage, the new `std::array` sprang into existence with contiguous storage, and a new library function `std::align` was added to help users parcel out aligned chunks of contiguous memory from large blocks of storage.

C++17 recognized that pointers are members of a larger class of contiguous iterators. N4284 “Contiguous iterators” defined the terms “contiguous iterator” [iterator.requirements.general]/6:

> Iterators that further satisfy the requirement that, for integral values `n` and dereferenceable iterator values `a` and `(a + n)`, `*(a + n)` is equivalent to `*(addressof(*a) + n)`, are called *contiguous iterators*. and “contiguous container” [container.requirements.general]/13: A *contiguous container* is

> a container that supports random access iterators and whose member types `iterator` and `const_iterator` are contiguous iterators.

Sadly, C++17 fell short of providing the tools necessary to make contiguous iterators usable in generic contexts. There is a set of concrete iterator types that are specified to be contiguous iterators, but there is no general mechanism in C++17 to determine if a given iterator type is a contiguous iterator. It is notable that N3884 "Contiguous Iterators: a refinement of random access iterators" *did* try to provide such a facility in the form of a new iterator tag `contiguous_iterator_tag`. LEWG rejected the new category tag in Issaquah 2014 out of fear that too much code would be broken by changing the iterator category of the existing contiguous iterators from `random_access_iterator_tag` to `contiguous_iterator_tag`. LEWG requested a new type trait be used to distinguish contiguous iterators. N4183 "Contiguous Iterators: Pointer Conversion & Type Trait" presented such a trait, but that proposal seems to have been regarded with indifference by LEWG.

This paper does not motivate the need for contiguous ranges, or for generic algorithms that work with contiguous ranges. The treatment in N3884 is sufficient.

Proposed Design

This paper proposes to take advantage of the opportunity provided by the distinct mechanisms in the Ranges TS to both leave the iterator category of the standard library's contiguous iterators unchanged *and* update the category of those iterators to denote them as contiguous. In the Ranges TS design, the category of iterators is determined using the `ranges::iterator_category_t` alias and its backing class template `ranges::iterator_category` (Ranges TS [iterator.assoc.types.iterator_category]). This trait is notably distinct from `std::iterator_traits` that determines the category of Standard iterators. By defining the Ranges and Standard traits to different categories, "new" code that is Ranges-aware can detect contiguous iterators using `ranges::iterator_category_t`, while "old" code that may break if presented a category other than random access for for e.g. `vector::iterator` will use `std::iterator_traits` and be presented `std::random_access_iterator_tag`.

This paper also proposes that contiguous iterators are a refinement of random access iterators as did N3884. Since a contiguous range `[i, s)` (where `s` is a sentinel for `i`) must be equivalent to some pointer range `[p, n)` and random access is certainly possible in that pointer range, it is trivial to implement random access for any contiguous iterator. Allowing contiguous iterators that are not random access would not admit any additional models or increase the expressive power of the design.

Unlike the prior art, this paper does not propose a customization point to retrieve the pointer value that corresponds to some contiguous iterator value. This paper's goal is to enable access to contiguous *ranges*, rather than provide mechanisms to handle individual contiguous iterators, and a sized contiguous range `rng` (or `[i, s)`) can always be converted to a pointer range `[data(rng), size(rng))` (resp. `[s - i ? addressof(*i) : nullptr, s - i)`). Implementation of generic algorithms that take advantage of contiguity does not require a facility to convert a "monopole" contiguous iterator to a pointer. The `copy` algorithm, for example, could be approximated with:

```
template<InputIterator I, Sentinel<I> S, Iterator O>
requires IndirectlyCopyable<I, O>
void copy(I i, S s, O o) {
  for (; i != s; ++i, ++o) {
    *o = *i;
  }
}

template<ContiguousIterator I, SizedSentinel<I> S, ContiguousIterator O>
requires IndirectlyCopyable<I, O> &&
  Same<value_type_t<I>, value_type_t<O>> &&
  is_trivially_copyable_v<value_type_t<O>>
```

```
void copy(I i, S s, O o) {
  if (s - i > 0) {
    std::memmove(&*o, &*i, (s - i) * sizeof(value_type_t<I>));
  }
}
```

Implementation Experience

A variant of this design has been implemented in CMCSTL2, the reference implementation of the Ranges TS, for several months.

Technical Specification

Wording relative to N4685, the Ranges TS Working Draft.

Change the synopsis of header `<experimental/ranges/iterator>` in [iterator.synopsis] as follows:

```
// [iterators.random.access], RandomAccessIterator:
template <class I>
concept bool RandomAccessIterator = see below;

// [iterators.contiguous], ContiguousIterator:
template <class I>
concept bool ContiguousIterator = see below;
```

[…]

```
// 9.6.3, iterator tags:
struct output_iterator_tag { };
struct input_iterator_tag { };
struct forward_iterator_tag : input_iterator_tag { };
struct bidirectional_iterator_tag : forward_iterator_tag { };
struct random_access_iterator_tag : bidirectional_iterator_tag { };
struct contiguous_iterator_tag : random_access_iterator_tag { };
```

Modify [iterator.assoc.types.iterator_category]/1 as follows:

1 `iterator_category_t<T>` is implemented as if:

[…]

```
template <class T>
struct iterator_category<T*>
  : enable_if<is_object<T>::value, ~~random_access~~contiguous_iterator_tag> { };
```

[…]

Insert a new paragraph after [iterator.assoc.types.iterator_category]/3:

-?- `iterator_category<I>::type` is `contiguous_iterator_tag` when `I` denotes:

- the member type `iterator` or `const_iterator` of a specialization of `std::basic_string` (ISO/IEC 14882:2014 [basic.string])
- the member type `iterator` or `const_iterator` of a specialization of `std::array` (ISO/IEC 14882:2014 [array])

- the member type `iterator` or `const_iterator` of a specialization of `std::vector` except for `std::vector<bool>` (ISO/IEC 14882:2014 [vector])

- the type returned by the `std::valarray` overloads of `std::begin` and `std::end` (ISO/IEC 14882:2014 [valarray.range])

[*Note:* Implementations of this document that also implement `basic_string_view` from ISO/IEC 14882:2017 are also encouraged to ensure that `iterator_category<I>::type` is `contiguous_iterator_tag` when `I` denotes the member type `iterator` or `const_iterator` of a specialization of `basic_string_view`.—*end note*]

Add a new subclause immediately after [iterators.random.access]:

9.3.?? Concept `ContiguousIterator` [iterator.contiguous]

The `ContiguousIterator` concept refines `RandomAccessIterator` and provides a guarantee that the denoted elements are stored contiguously in memory.

```
template <class I>
concept bool ContiguousIterator =
  RandomAccessIterator<I> &&
  DerivedFrom<iterator_category_t<I>, contiguous_iterator_tag> &&
  is_lvalue_reference<reference_t<I>>::value &&
  Same<value_type_t<I>, remove_cv_t<remove_reference_t<reference_t<I>>>>;
```

Let `a` and `b` be a dereferenceable iterators of type `I` such that `b` is reachable from `a`. `ContiguousIterator<I>` is satisfied only if:

- `addressof(*(a + (b - a))) == addressof(*a) + (b - a)`

Modify [std.iterator.tags]/1 as follows:

1 […] The category tags are: `input_iterator_tag`, `output_iterator_tag`, `forward_iterator_tag`, `bidirectional_iterator_tag`, ~~and~~ random_access_iterator_tag, and `contiguous_iterator_tag`. […]

```
namespace std { namespace experimental { namespace ranges { inline namespace v1 {
  struct output_iterator_tag { };
  struct input_iterator_tag { };
  struct forward_iterator_tag : input_iterator_tag { };
  struct bidirectional_iterator_tag : forward_iterator_tag { };
  struct random_access_iterator_tag : bidirectional_iterator_tag { };
  struct contiguous_iterator_tag : random_access_iterator_tag { };
}}}}
```

Modify the synopsis of class template `reverse_iterator` in [reverse.iterator] as follows:

```
[…]
template <BidirectionalIterator I>
class reverse_iterator {
public:
  using iterator_type = I;
  using difference_type = difference_type_t<I>;
  using value_type = value_type_t<I>;
  using iterator_category = ~~iterator_category_t<I>~~see below;
```

Add a new paragraph following the synopsis:

-?- The member typedef-name `iterator_category` denotes `random_access_iterator_tag` if `ContiguousIterator<I>` is satisfied, and otherwise denotes the same type as `iterator_category_t<I>`.

Modify the synopsis of `<experimental/ranges/range>` in [range.synopsis] as follows:

[…]

```
  // 10.6.10, RandomAccessRange:
  template <class T>
  concept bool RandomAccessRange = see below;

  // 10.6.??, ContiguousRange:
  template <class T>
  concept bool ContiguousRange = see below;
}}}}
```

Modify [range.primitives.data/1] as follows:

[…]

(1.3) — Otherwise, `ranges::begin(E)` if it is a valid expression of pointer to object type.

(1.?) — Otherwise, if `ranges::begin(E)` is a valid expression whose type satisfies `ContiguousIterator`, `ranges::begin(E) == ranges::end(E) ? nullptr : addressof(*ranges::begin(E))`, except that `E` is evaluated only once.

(1.4) — Otherwise, `ranges::data(E)` is ill-formed.

[…]

Add a new subclause after [ranges.random.access]:

10.6.?? Contiguous ranges [ranges.contiguous]

1 The `ContiguousRange` concept specifies requirements of a `RandomAccessRange` type for which `begin` returns a type that satisfies `ContiguousIterator` ([iterators.contiguous]).

```
template <class R>
concept bool ContiguousRange =
  Range<R> && ContiguousIterator<iterator_t<R>> &&
  requires(R& r) {
    ranges::data(r);
    requires Same<decltype(ranges::data(r)),
add_pointer_t<reference_t<iterator_t<R>>>>;
  };
```